

Design and Implementation of a Real-Time Water Quality Monitoring System Using FIWARE

Abstract

Environmental monitoring often relies on periodic sampling, producing infrequent and retrospective data. A further challenge is the semantic incompatibility of proprietary monitoring infrastructure, limiting data integration and exchange across institutional boundaries. This project investigated the application of FIWARE and the NGSI-LD standard to address both challenges through the design and implementation of a real-time water quality monitoring system, contributing to a broader initiative to establish a federated water data space.

The system was implemented using Orion-LD as the NGSI-LD context broker, QuantumLeap and TimescaleDB for time-series data persistence, and a React web frontend providing real-time and historical data visualisation, map-based navigation, and entity management. A WFD-aligned entity hierarchy was implemented using Smart Data Models, and system viability was demonstrated through simulation of real-time sensor updates derived from historical Environment Agency data.

All project objectives were successfully achieved, with the results demonstrating that NGSI-LD and linked data principles provide a practical foundation for semantic interoperability in environmental monitoring, enabling meaningful data exchange across institutional boundaries without requiring bespoke integration layers. The system thus represents a viable contribution to the development of a federated data space, consistent with approaches demonstrated by the AD4GD and WATERVERSE projects.

Student Name: Peter Jackson

Student ID: 32031854

Supervisor: Dr. Carlos Eduardo da Silva

Word Count: 3300

Table of Contents

1. Introduction.....	3
2. Literature Review.....	4
2.1 Challenges in Environmental Monitoring.....	4
2.2 IoT Architectures for Environmental Monitoring.....	4
2.3 Context Management and Publish-Subscribe Architectures.....	4
2.4 IoT Platform Approaches and Semantic Interoperability.....	5
2.5 Standardised Data Models and Federated Data Spaces.....	5
2.6 Visualisation of Environmental Data.....	6
3. Proposed Solution.....	6
4. Engineering Approaches and Technologies.....	7
4.1 Development Methodology and Tooling.....	7
4.2 FIWARE and the NGSI-LD Standard.....	8
4.3 Time-Series Data Persistence.....	8
4.4 Infrastructure and Containerisation.....	8
4.5 Backend Architecture.....	9
4.6 Frontend Architecture.....	9
4.7 Testing Approach and Tooling.....	9
5. Ethical and Professional Issues.....	10
5.1 Data Accuracy and Reliability.....	10
5.2 Accessibility and Inclusive Design.....	10
5.3 Open Data and Professional Responsibility.....	10
6. Conclusion.....	11
References.....	12
Appendix A: Transparency Declaration Statement.....	14
Appendix B: List of Abbreviations.....	14
Appendix C: Development References.....	15
Appendix D: Test Cases.....	16
Functional and Integration Testing.....	16
Accessibility Testing.....	16
Appendix E: Container Diagram.....	17

1. Introduction

Effective water quality monitoring is essential for environmental protection and regulatory compliance. In the United Kingdom, it is a statutory obligation under The Water Environment (Water Framework Directive) (England and Wales) Regulations 2017 (SI 2017/407) to establish monitoring programmes, providing “a coherent and comprehensive overview of water status” (reg. 11). In practice this is commonly achieved through periodic sampling. Analysis of Environment Agency water records for the Sheffield area indicates that historic records are stored at approximately monthly intervals (Environment Agency, 2026), producing infrequent, retrospective data that limits the ability to respond to rapid changes in water quality.

A further challenge is the inability to share and integrate environmental monitoring data across institutional boundaries due to proprietary systems using bespoke technologies. Addressing this requires a standardised, semantically interoperable approach to data representation.

Advances in IoT technologies and open smart city platforms present an opportunity to address both challenges. With standards such as NGSI-LD, sensor data can be represented using semantically rich, interoperable linked data models. This allows for the construction of federated data spaces where environmental data is standardised and consistently interpreted across institutional boundaries.

This project was motivated by prior professional exposure to industrial monitoring and SCADA systems during a placement year, which highlighted the value of real-time operational data and the complexity of the infrastructure required to deliver it reliably in regulated environments.

This project investigates the application of FIWARE and NGSI-LD to design a real-time water quality monitoring system, contributing to a broader data space initiative.

The project addresses the following objectives:

1. Design and implement NGSI-LD data models representing a WFD-aligned hierarchy of water bodies, monitoring stations and water quality observations.
2. Develop a pipeline to support real-time data monitoring using FIWARE and supporting time-series technologies.
3. Build a web-based dashboard providing hierarchical navigation of monitoring stations, current parameter readings, and historical time-series visualisation.
4. Demonstrate system viability through simulation of real-time sensor updates derived from historical Environment Agency data.

2. Literature Review

2.1 Challenges in Environmental Monitoring

Environmental monitoring systems frequently rely on heterogeneous, proprietary sensor equipment, creating persistent challenges for interoperability in the analysis of data captured from different types of sensors (Ullo & Sinha, 2020). The absence of shared data formats and vocabularies means data produced by one system is frequently incompatible with another, presenting a significant barrier to large-scale environmental analysis across institutional boundaries. Addressing these challenges requires continuous sensing infrastructure and standardised approaches to data representation.

2.2 IoT Architectures for Environmental Monitoring

Advances in Internet of Things (IoT) technologies have significantly expanded the potential of environmental monitoring; allowing for continuous, networked sensor deployments capable of transmitting measurements in real time across geographically distributed sites. Atzori et al. (2010) describe the IoT paradigm as a result of the convergence of three visions: things-oriented, internet-oriented and semantic-oriented; spanning physical object identification and traceability, network connectivity, and knowledge representation. However, the same authors identify scalability and adaptability to heterogeneous technologies as crucial challenges, noting that IoT systems rely on a vast set of objects each accessible through its own dialect, creating the need for an abstraction layer capable of harmonising access through a common language and procedure (Atzori et al., 2010).

As sensor deployments grow in scale, the traditional approach of connecting sensors directly to applications individually becomes infeasible (Perera et al., 2014). Middleware solutions have been developed in response, decoupling data producers from consumers and separating raw data production from higher-level data consumption. However, middleware alone does not resolve interoperability at the semantic level; without a common data model and shared vocabulary, data produced by one system may remain opaque to another, regardless of the middleware layer in place.

2.3 Context Management and Publish-Subscribe Architectures

The concept of context provides a useful abstraction for managing the dynamic state of distributed IoT systems. Dey (2001) defines context as any information that can be used to characterise the situation of an entity, where entities may be a person, place or object. In an environmental monitoring system, context may include sensor readings, geo-spatial coordinates, timestamps, and the semantic relationships between entities such as monitoring stations and the water bodies they monitor. Crucially, context is not equivalent to raw data: it is structured, maintained and queryable, providing a stable interface for applications to access current state independently of underlying sensor implementations.

This relational dimension is a core requirement of context modelling, as context-aware applications must capture the various relationships and dependencies that exist among contextual information (Bettini et al., 2010), a requirement formally addressed in the NGSI-LD specification, which

models context as a property graph of entities connected by typed relationships and properties (ETSI, 2025).

The dissemination of such context commonly relies on the publish-subscribe pattern. Eugster et al. (2003) establish publish-subscribe as a mechanism for decoupled communication, in which producers publish updates without direct knowledge of consumers, and subscribers receive notifications asynchronously based on their registered interests. This decoupling increases scalability by removing explicit dependencies between participants, reducing coordination overhead and making the communication infrastructure well-suited to distributed, asynchronous environments (Eugster et al., 2003). In an environmental monitoring system, this pattern enables real-time propagation of sensor readings to multiple downstream services, such as time-series databases and monitoring dashboards, without tight coupling between components.

2.4 IoT Platform Approaches and Semantic Interoperability

A range of platforms implement context management and publish-subscribe architectures for IoT deployments. Commercial offerings such as AWS IoT Core and Azure IoT Hub provide managed broker infrastructure with broad protocol support and enterprise integration capabilities (Panagou et al., 2025). While this protocol-level support facilitates connectivity across heterogeneous devices, interoperability at the data representation level remains a challenge, particularly in multi-institutional deployments where differences in platform-specific data models and APIs limit seamless data exchange.

FIWARE addresses this through its adoption of NGSI-LD, a standard API for publishing, querying, and subscribing to context information using JSON-LD-based linked data representation (ETSI, 2025). Linked data allows attribute semantics to be defined externally via context documents, as opposed to being implicitly embedded within application logic. An entity produced by one NGSI-LD system can therefore be directly interpreted by any other conformant system. The FIWARE Foundation maintains an open-source implementation of the NGSI-LD standard through the Orion-LD context broker, as part of a broader ecosystem of interoperable components (FIWARE Foundation, n.d.-a).

2.5 Standardised Data Models and Federated Data Spaces

Semantic interoperability at the API level is further strengthened by the use of standardised entity schemas. The Smart Data Models initiative, a program led by FIWARE, IUDX, TM Forum, OASC and others, supports the adoption of compatible common data models that underpin a digital market of interoperable and replicable smart solutions in multiple sectors (FIWARE Foundation, n.d.-c).

Crettaz & Bréline (2024) demonstrate this approach in the AD4GD project, where FIWARE context brokers, NGSI-LD, and Smart Data Models including ‘WaterQualityObserved’ schemas form part of the heterogeneous IoT data integration architecture for the EU Green Deal Data Space. This showcases how shared data models facilitate interoperability and integration of heterogeneous IoT data sources.

WATERVERSE, an EU Horizon Europe-funded project developing a Water Data Management Ecosystem for the water sector, adopts FIWARE building blocks as the basis for its Water Data Space infrastructure, involving water sector stakeholders across six EU countries (WATERVERSE, n.d.). The project demonstrates the growing institutional adoption of open standards-based platforms for federated water data infrastructure.

Data spaces are federated ecosystems in which data is shared securely and efficiently among trusted partners, based on common rules and standards (European Data Portal, 2025). They require participants to produce and consume mutually interpretable data, making semantic interoperability foundational rather than optional. As the preceding examples illustrate, open standards such as NGSI-LD and shared data models are increasingly recognised as the technical foundation enabling this requirement to be met in practice.

2.6 Visualisation of Environmental Data

Effective presentation of environmental data requires attention to both information design and accessibility. Research into environmental analytics dashboards identifies visual design as the primary determinant of a dashboard's effectiveness, with appropriate visualisation techniques as key subfactors (Mohd Yusof & Maarof, 2022). For threshold-based data such as the WFD ecological status classifications, colour encoding is of particular significance: indicators of ecological status must be unambiguously distinguishable across different viewing conditions for all users.

Tol (2021) provides schemes explicitly with scientific data visualisation in mind, noting that effective colour choices must be distinct for colour-blind readers, distinguishable from black and white, and appropriate for the type of data being represented – whether qualitative, diverging or sequential. These considerations directly informed the colour choices made in this system's interface design.

3. Proposed Solution

The preceding literature review establishes the absence of shared data models and vocabularies as the primary barrier to semantic interoperability in environmental monitoring systems. Middleware solutions alone are insufficient; data must be represented using an agreed, externally defined vocabulary to enable meaningful exchange across institutional boundaries.

The proposed solution addresses this through two complementary components: a context management layer built on FIWARE's Orion-LD context broker implementing the NGSI-LD standard, along with a set of domain-specific data models drawn from the Smart Data Models programme. Together these provide the semantic interoperability required to contribute to a federated water data space.

The following specifications were defined for the proposed solution:

1. The solution shall represent water quality monitoring entities using NGSI-LD data models aligned with the WFD classification hierarchy of water bodies, monitoring stations, and observations.
2. The solution shall provide a pipeline capable of ingesting real-time water quality observations and storing them to a time-series database.
3. The solution shall provide a web-based dashboard supporting station navigation and display of both current and historical readings.
4. The solution shall provide a map-based interface allowing spatial navigation and area-based selection of monitoring stations.
5. The solution shall provide a graphical interface for the creation and management of NGSI-LD entities without requiring direct API interaction.
6. The solution shall conform to WCAG accessibility guidelines, including the use of colour-blind safe colour schemes.
7. System viability shall be demonstrated through simulation of real-time sensor updates derived from historical Environment Agency data.

4. Engineering Approaches and Technologies

The engineering approaches chosen for this project were driven by the requirement to support a federated environmental data space, necessitating semantic interoperability through linked data technologies.

4.1 Development Methodology and Tooling

An agile style development methodology was adopted for this project. Given the exploratory nature of the project, weekly iterative sprints enabled continuous evaluation and refinement. Beck et al. (2001) establish agile as the prioritisation of working software and responsiveness to change over rigid planning, properties well-suited for a single-developer project with evolving requirements.

Tasks were managed using a Trello board, chosen over Jira due to its lightweight card-based model; Jira's configuration and issue tracking features are designed for more complex, cross-functional team projects and are not required for a single-developer project (Atlassian, n.d.-a).

Version control was managed using Git, with the remote repository hosted on GitHub. Git is the most widely used modern version control system in the world, relied upon by a vast number of commercial and open-source projects and is well supported across operating systems and IDEs (Atlassian, n.d.-b). GitHub was selected over alternatives such as Bitbucket or Codeberg due to its ease of use and strong integration with the project's development tooling.

Neovim was used as the primary development environment, configured with LSP support for Python and JavaScript, the project's primary languages, enabling autocompletion, linting, and code navigation. Neovim operates directly in the terminal, integrating seamlessly into a tmux-based

workflow alongside running services and Docker containers. IntelliJ IDEA was the main alternative considered but was not chosen, as a lighter, more configurable environment was preferred.

4.2 FIWARE and the NGSI-LD Standard

The selection of FIWARE and NGSI-LD is directly motivated by the project's role within a broader initiative to develop a federated water data space, as described in Section 2.5. This requires semantically interoperable data exchange between participating systems. NGSI-LD was selected for its use of JSON-LD and linked data principles, which enable interoperability through a shared semantic model in which entity types and attributes are mapped to globally unique URI-based identifiers via `@context` documents (ETSI, 2025). NGSI-v2 was considered but ultimately not chosen, as its flat JSON structure relies on semantics defined at the application level rather than externally via context documents, limiting interoperability and preventing effective contribution to a federated data space.

FIWARE was chosen over managed IoT broker services such as AWS IoT Core and Azure IoT Hub as these do not implement the NGSI-LD standard. Data represented using their proprietary device models such as AWS Device Shadows or Azure Device Twins cannot be directly consumed by NGSI-LD-compliant systems without additional transformation layers, thus increasing complexity and undermining the federated data space requirement.

The discontinuation of IBM Watson IoT Platform in December 2023 without a direct replacement (Sharwood, 2022) further illustrates the vendor lock-in risk with proprietary IoT infrastructure. Within FIWARE, Orion-LD was selected as the context broker due to its support for the NGSI-LD API and active maintenance within the FIWARE ecosystem (FIWARE Foundation, n.d.-a).

4.3 Time-Series Data Persistence

The context broker (Orion-LD) only maintains the current state of entities and provides no historical data persistence. QuantumLeap, a FIWARE Generic Enabler, addresses this by subscribing to NGSI-LD notifications and persisting observations to a time-series database (FIWARE Foundation, n.d.-b). TimescaleDB was selected as the database over CrateDB as it is an open-source PostgreSQL extension designed for real-time analytics on time-series data while maintaining full SQL support (Timescale, n.d.). CrateDB's distributed architecture introduces unnecessary operational complexity for this project's scale.

4.4 Infrastructure and Containerisation

The FIWARE service stack, consisting of Orion-LD, MongoDB, QuantumLeap, and TimescaleDB, was containerised using Docker Compose. Containerisation ensures consistent, reproducible deployment across development environments and eliminates dependency conflicts between components, with specific version and dependency requirements (Merkel, 2014). Docker provides process-level isolation with significantly lower resource overhead than virtual machine-based

approaches. The FastAPI backend and React frontend were run locally during development; a production deployment would containerise these along with the rest of the stack.

Docker Compose was selected as it allows all services to be defined declaratively in a single configuration file and started with a single command, reducing the operational complexity of running a distributed system locally. Cloud deployment via AWS or Azure was considered but discounted due to cost and networking configuration overhead, though it would be appropriate for a production system requiring high availability and scalability.

4.5 Backend Architecture

A FastAPI backend was developed to expose water quality data and entity management operations to the frontend via a RESTful API. FastAPI was chosen over alternatives including Flask and Django due to its support for asynchronous request handling, well-suited for concurrent I/O operations such as database queries and API calls (Tiangolo, n.d.). It also auto-generates OpenAPI documentation via its built-in Swagger UI, which proved helpful for testing during development. FastAPI is an actively maintained and increasingly adopted framework, recording one of the largest year-on-year increases among web frameworks (Stack Overflow, 2025). Flask was deemed less suitable due to its minimal and unopinionated structure, requiring additional effort to structure a scalable multi-router application. Django was considered unnecessarily complex for this project, as its built-in ORM and administrative features were not required given that data models are managed externally by the FIWARE components.

4.6 Frontend Architecture

The frontend was implemented as a React single-page application, built with Vite. React was preferred over alternatives such as Vue and Svelte due to its larger ecosystem and widespread industry adoption; it remains one of the most widely used frontend frameworks among professional developers (Stack Overflow, 2025). Vite was chosen over Create React App due to its significantly faster development server and hot module replacement, reducing iteration time during development (Vite, n.d.). Tailwind CSS was selected over Bootstrap as its framework provides flexible, low-level control over interface design through composable utility classes (Tailwind Labs, n.d.), supporting the custom dashboard requirements of the system. For data visualisation, Recharts was chosen over Chart.js and D3. Recharts is a composable charting library built on React components, making it easy to integrate (Recharts, n.d.). D3 offers greater flexibility but requires substantially more effort implementing standard chart types, making it less suitable for the project's time constraints and scope. The Paul Tol colour scheme was applied throughout to satisfy the accessibility requirements outlined in Section 5.2.

4.7 Testing Approach and Tooling

A manual testing approach was adopted rather than a formal automated test suite given the project's scale. API endpoints were tested using FastAPI's built-in Swagger UI, and the NGSI-LD pipeline

was verified using curl, confirming that entity creation triggered QuantumLeap subscriptions and correct persistence in TimescaleDB, validated with direct SQL queries. Frontend integration was verified by exercising the full user journey through the dashboard. Accessibility was audited using Google Lighthouse against WCAG criteria. A production deployment or larger-scale system would warrant automated integration testing and CI/CD pipelines.

5. Ethical and Professional Issues

The development of a water quality monitoring system raises several ethical and professional considerations that directly influenced design decisions throughout the project.

5.1 Data Accuracy and Reliability

Environmental monitoring systems inform assessments and, in production deployments, influence decisions affecting public health and ecosystem management; accuracy and reliability are therefore of utmost importance.

This system is designed to handle real-time ingestion of sensor data; the viability of this is demonstrated using simulated data derived from historical Environment Agency observations. To mitigate the risk of misrepresentation, the simulated nature of the demonstration is explicitly documented.

5.2 Accessibility and Inclusive Design

As a system presenting environmental data to potentially diverse stakeholders, accessibility is an ethical obligation rather than an optional enhancement, as failure to address it risks excluding users from accessing environmental information. As such, WCAG compliance was adopted as a design requirement. The Paul Tol bright colour scheme was chosen to ensure data visualisations remain distinguishable for users with colour vision deficiencies (Tol, 2021). Google Lighthouse was used to audit the frontend against WCAG standards (Google, n.d.), with any failures treated as defects requiring resolution during development.

5.3 Open Data and Professional Responsibility

The system uses public Environment Agency water quality data published under the Open Government Licence, permitting reuse with attribution. No personal data is collected, processed or stored, therefore preserving privacy and eliminating GDPR obligations. More broadly, development was conducted with reference to professional obligations to act in the public interest, reflected in the use of established open standards (NGSI-LD, Smart Data Models) rather than ad hoc solutions.

6. Conclusion

This project addressed the challenge of achieving semantic interoperability in environmental monitoring systems, where heterogeneous and proprietary technologies limit data integration across institutional boundaries. It investigated whether the use of FIWARE and the NGSI-LD standard could support the development of a real-time water quality monitoring system capable of contributing to a federated data space.

All project objectives were successfully achieved, with the resulting system demonstrating the viability of the proposed solution. A set of NGSI-LD data models aligned with the WFD hierarchy was implemented, representing water bodies, monitoring stations, and observations. A data pipeline was developed using FIWARE components, enabling ingestion, contextualisation, and persistence of real-time sensor data via a time-series database. This was complemented by a web-based dashboard supporting real-time and historical data visualisation, map-based navigation, and entity management. System viability was demonstrated through simulation of real-time sensor updates derived from historical Environment Agency data, validating the end-to-end pipeline and dashboard functionality.

These findings support the conclusion that NGSI-LD and linked data principles provide a practical and effective foundation for semantic interoperability in environmental monitoring systems. By externalising data semantics through shared data models, this approach enables meaningful data exchange across institutional boundaries without requiring bespoke integration layers. This is consistent with the approaches demonstrated by the AD4GD and WATERVERSE projects discussed in Section 2.5, reinforcing this solution as a viable contribution to the development of a federated data space.

References

- Atlassian. (n.d.-a). *Trello vs. Jira: Which to choose*. Retrieved from <https://www.atlassian.com/blog/project-management/trello-vs-jira>
- Atlassian. (n.d.-b). *What is Git*. Retrieved from <https://www.atlassian.com/git/tutorials/what-is-git>
- Atzori, L., Iera, A., & Morabito, G. (2010). The Internet of Things: A survey. *Computer Networks*, 54(15), 2787–2805. <https://doi.org/10.1016/j.comnet.2010.05.010>
- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Manifesto for Agile Software Development*. Retrieved from <https://agilemanifesto.org>
- Bettini, C., Brdiczka, O., Henriksen, K., Indulska, J., Nicklas, D., Ranganathan, A., & Riboni, D. (2010). A survey of context modelling and reasoning techniques. *Pervasive and Mobile Computing*, 6(2), 161–180. <https://doi.org/10.1016/j.pmcj.2009.06.002>
- Crettaz, C., & Bréchine, A. (2024). *AD4GD D3.1: Heterogeneous IoT data integration model (V1.2)*. Zenodo. <https://doi.org/10.5281/zenodo.13757222>
- Dey, A. K. (2001). Understanding and using context. *Personal and Ubiquitous Computing*, 5(1), 4–7. <https://doi.org/10.1007/s007790170019>
- Environment Agency. (2026). *Water quality sampling data: River Sheaf catchment* [Data set]. Retrieved from <https://environment.data.gov.uk/water-quality/>
- ETSI. (2025). *Context Information Management (CIM); NGSI-LD API (ETSI GS CIM 009 V1.9.1)*. European Telecommunications Standards Institute. Retrieved from https://www.etsi.org/deliver/etsi_gs/CIM/001_099/009/01.09.01_60/gs_CIM009v010901p.pdf
- Eugster, P. T., Felber, P. A., Guerraoui, R., & Kermarrec, A.-M. (2003). The many faces of publish/subscribe. *ACM Computing Surveys*, 35(2), 114–131. <https://doi.org/10.1145/857076.857078>
- European Data Portal. (2025, June 11). *Interoperability in data spaces: Building Europe's digital future*. data.europa.eu. <https://data.europa.eu/en/news-events/news/interoperability-data-spaces-building-europes-digital-future>
- FIWARE Foundation. (n.d.-a). *context.Orion-LD*. GitHub. Retrieved from <https://github.com/FIWARE/context.Orion-LD>
- FIWARE Foundation. (n.d.-b). *FIWARE catalogue*. GitHub. Retrieved from <https://github.com/FIWARE/catalogue>
- FIWARE Foundation. (n.d.-c). *Smart Data Models initiative*. Retrieved from <https://www.fiware.org/smart-data-models/>
- Google. (n.d.). *Lighthouse accessibility scoring*. Chrome for Developers. Retrieved from <https://developer.chrome.com/docs/lighthouse/accessibility/scoring>

- Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239). Retrieved from <https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment>
- Mohd Yusof, N., & Maarof, M. A. (2022). Visual design as the key factor for effective environmental analytics dashboard. *Proceedings*, 82(1), 21. <https://doi.org/10.3390/proceedings2022082021>
- Panagou, I. C., Katsoulis, S., Nannos, E., Zantalis, F., & Koulouras, G. (2025). A comprehensive evaluation of IoT cloud platforms: A feature-driven review with a decision-making tool. *Sensors*, 25(16), 5124. <https://doi.org/10.3390/s25165124>
- Perera, C., Zaslavsky, A., Christen, P., & Georgakopoulos, D. (2014). Context aware computing for the Internet of Things: A survey. *IEEE Communications Surveys & Tutorials*, 16(1), 414–454. <https://doi.org/10.1109/SURV.2013.042313.00197>
- Recharts. (n.d.). *Recharts: Redefined chart library built with React and D3*. Retrieved from <https://recharts.org/>
- Sharwood, S. (2022, November 15). Watson IoT Platform to exit IBM's cloud. *The Register*. https://www.theregister.com/2022/11/15/ibm_set_to_retire_watson
- Stack Overflow. (2025). *Stack Overflow Developer Survey 2025*. Retrieved from <https://survey.stackoverflow.co/2025/>
- Tailwind Labs. (n.d.). *Tailwind CSS*. Retrieved from <https://tailwindcss.com/>
- Tiangolo. (n.d.). *Concurrency and async / await*. FastAPI. Retrieved from <https://fastapi.tiangolo.com/async/>
- Timescale. (n.d.). *timescaledb*. GitHub. Retrieved from <https://github.com/timescale/timescaledb>
- Tol, P. (2021, August 18). *Colour schemes*. SRON. Retrieved from <https://personal.sron.nl/~pault/data/colourschemes.pdf>
- Ullo, S. L., & Sinha, G. R. (2020). Advances in smart environment monitoring systems using IoT and sensors. *Sensors*, 20(11), 3113. <https://doi.org/10.3390/s20113113>
- Vite. (n.d.). *Vite: Next generation frontend tooling*. Retrieved from <https://vitejs.dev/>
- The Water Environment (Water Framework Directive) (England and Wales) Regulations 2017, SI 2017/407. (2017). Retrieved from <https://www.legislation.gov.uk/uksi/2017/407/contents>
- WATERVERSE. (n.d.). *The project*. Retrieved from <https://waterverse.eu/the-project/>

Appendix A: Transparency Declaration Statement

AI was used in line with AITS Level 2 as permitted for this assessment. AI tools were used for the discrete and specific goal of assisting in shaping the structure of the academic report to aid in a clear narrative thread through the report. All AI generated content were taken strictly as ideas and have been refined and reviewed.

Appendix B: List of Abbreviations

Abbreviation	Full Term
API	Application Programming Interface
CI/CD	Continuous Integration/Continuous Deployment
GDPR	General Data Protection Regulation
IDE	Integrated Development Environment
IoT	Internet of Things
JSON-LD	JavaScript Object Notation – Linked Data
LSP	Language Server Protocol
NGSI-LD	Next Generation Service Interfaces – Linked Data
ORM	Object-Relational Mapping
SCADA	Supervisory Control and Data Acquisition
SI	Statutory Instrument
SQL	Structured Query Language
UI	User Interface
URI	Uniform Resource Identifier
WCAG	Web Content Accessibility Guidelines
WFD	Water Framework Directive

Appendix C: Development References

The following references were utilised during the development of the Real-Time Water Quality Monitoring System:

asyncpg. (n.d.). *asyncpg documentation*. Retrieved from <https://magicstack.github.io/asyncpg/current/>

Axios. (n.d.). *Axios documentation*. Retrieved from <https://axios-http.com/docs/intro>

Environment Agency. (n.d.). *Water quality data archive API*. Retrieved from <https://environment.data.gov.uk/water-quality/>

FastAPI. (n.d.). *FastAPI documentation*. Retrieved from <https://fastapi.tiangolo.com/>

FIWARE Foundation. (n.d.). *NGSI-LD step-by-step tutorials*. Read the Docs. Retrieved from <https://ngsi-ld-tutorials.readthedocs.io/en/latest/>

FIWARE Foundation. (n.d.). *WaterQualityObserved smart data model specification*. GitHub. Retrieved from <https://github.com/smart-data-models/dataModel.WaterQuality/blob/master/WaterQualityObserved/doc/spec.md>

Franklin, W. R. (n.d.). *PNPOLY — Point Inclusion in Polygon Test*. Retrieved from https://wrfranklin.org/Research/Short_Notes/pnpoly.html

IBM. (n.d.). *IBM Plex typeface*. Retrieved from <https://www.ibm.com/plex/>

Leaflet. (n.d.). *Leaflet: An open-source JavaScript library for interactive maps*. Retrieved from <https://leafletjs.com/>

Orion-LD. (n.d.). *Orion-LD context broker documentation*. GitHub. Retrieved from <https://github.com/FIWARE/context.Orion-LD/tree/develop/doc/manuals-ld>

QuantumLeap. (n.d.). *QuantumLeap documentation*. Read the Docs. Retrieved from <https://quantumleap.readthedocs.io/en/latest/>

React. (n.d.). *React documentation*. Retrieved from <https://react.dev/>

React Router. (n.d.). *React Router documentation*. Retrieved from <https://reactrouter.com/>

Structurizr. (n.d.). *Structurizr: Software architecture diagrams as code*. Retrieved from <https://structurizr.com/>

Appendix D: Test Cases

Functional and Integration Testing

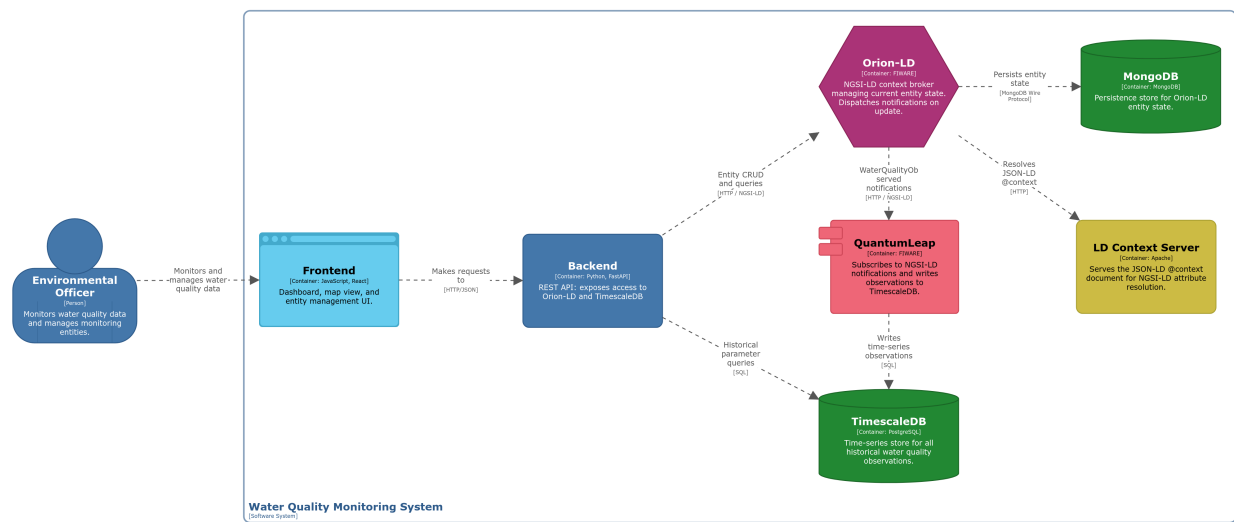
ID	Description	Method	Expected Result	Result
TC01	All stations load on dashboard	Swagger UI: GET /api/stations	JSON array of stations returned	PASS
TC02	Latest readings for a station	Swagger UI: GET /api/stations/{id}/latest	Latest parameter readings returned	PASS
TC03	Historical time-series query	Swagger UI: GET /api/stations/{id}/history/{param}	Timestamped data array returned from TimescaleDB	PASS
TC04	CSV ingestion pipeline	Upload via Entity Manager UI	Data appears in station dashboard	PASS
TC05	History window selector	Manual: browser UI	Chart updates to reflect selected time range	PASS
TC06	Entity creation (station)	Manual: Entity Manager form	Station appears in entity list and Orion-LD	PASS
TC07	Entity deletion	Manual: Entity Manager delete	Station removed from list and Orion-LD	PASS
TC08	NGSI-LD subscription	Curl POST to Orion-LD	Row written confirming subscription triggered	PASS
TC09	Polygon station selection	Manual: draw polygon on map page	Correct stations selected within boundary	PASS

Accessibility Testing

All frontend pages were audited using Google Lighthouse, with a goal score of at least 90/100.

ID	Page	Score	Result
TC10	Dashboard	91	PASS
TC11	Map	96	PASS
TC12	Entity Manager	92	PASS

Appendix E: Container Diagram



Container View: Water Quality Monitoring System
Level 2 — Containers
Saturday, April 16, 2026 at 6:56 PM British Summer Time

A full resolution version of this diagram is shown in the demonstration of my solution (Video 1).